CISC3024 Pattern Recognition

AI Assignment #2 — Probability / Bayes theory

ABNN: Make Me a BNN — converting a pretrained DNN into a Bayesian network (CVPR 2024), reproduced on CIFAR-10

Name: HOI KAI CHENG Student ID: UC325381

AI tool used: Cursor IDE in agent mode (model: Cursor Grok 4.6). As required by the assignment, every line of code, every experiment and this report were produced by the AI agent; my own contribution was limited to writing prompts and making the choices recorded in Section 1.

1. How I asked the AI tool to find the algorithm

I used the agent mode of the Cursor IDE, which can read files, run shell commands, write code and search the web, so a single conversation could carry the whole assignment from literature search to report. My first message attached the assignment PDF and the list of required report sections:

```
d:\UM\Year4\CISC3024\AI Assignment2\AIassign2.pdf
Your AI assignment report in Word/PDF format should contain the following details
(1) how you ask AI tools to find the algorithm, (2) algorithm description,
(3) how AI implements the algorithm, (4) experiment settings and results,
(5) what you have learnt from this AI assignment, and (6) a webpage link of your
source codes.
```

The agent read the PDF and extracted the constraints: a **recent classification / recognition algorithm based on probability or Bayes theory** for computer vision and pattern recognition; the report must document the six numbered sections; and **I am not allowed to write any code myself**. It also inspected the previous assignment (ConvNeXt V2 on CIFAR-10 in the same GitHub repository), the machine (Python 3.12, PyTorch 2.13 CPU-only, 20 logical cores) and then searched the web for recent peer-reviewed methods rather than guessing. The shortlist it came back with was:

Candidate	Venue / year	Why it was considered	Why it was or was not chosen
ABNN — "Make Me a BNN" (recommended)	CVPR 2024	Explicitly Bayesian: a post-hoc BNN obtained by attaching Gaussian noise to normalisation layers; official CIFAR-10 classification + SVHN OOD protocol.	Chosen: most recent, clearly Bayesian, and feasible on CPU.
Deep Deterministic Uncertainty (DDU)	CVPR 2023	Class-conditional Gaussians (GDA) on last-layer features; a classical Bayes classifier on top of a CNN.	Simpler and faster, but less "Bayesian neural network" than ABNN.
Fisher-information evidential DL (I-EDL)	ICML 2023	Predicts Dirichlet evidence and therefore a full categorical posterior.	Strong probabilistic story, but a heavier training recipe than ABNN.

The agent asked me to pick one of those three. I chose **ABNN (CVPR 2024)** and told it to keep going, to put the source on `https://github.com/HOIKAICHENG/CISC3024.git`, and to sign the report with my name (HOI KAI CHENG) and student ID (UC325381). From that point the

agent implemented the paper from scratch, ran the experiments and wrote this report. I did not edit any source file by hand.

Reference. G. Franchi, O. Laurent, M. Leguery, A. Bursuc, A. Pilzer and A. Yao, "Make Me a BNN: A Simple Strategy for Estimating Bayesian Uncertainty from Pre-trained Models", CVPR 2024 (arXiv:2312.15297).

2. Algorithm description

2.1 Motivation

A standard deep network keeps a single weight vector ω_{MAP} and treats it as an oracle. That point estimate is often over-confident: a cat that the model has never seen, or a house-number image when the training set was CIFAR-10, can still receive a softmax probability close to one. Bayesian neural networks (BNNs) instead try to represent the posterior $P(\omega \mid \mathcal{D})$ and to predict by averaging,

$$P(y \mid x, \mathcal{D}) = \int P(y \mid x, \omega) P(\omega \mid \mathcal{D}) d\omega \quad (2)$$

which in practice is replaced by a finite average over M weight samples. Classical variational BNNs put a Gaussian on every weight and are unstable to train on modern ConvNets. Deep ensembles work well but multiply the training cost. Last-layer Laplace methods are cheap but only see high-level features. ABNN sits between these extremes: start from a finished deterministic network and turn **only the normalisation layers** into a cheap Bayesian approximation.

2.2 From MAP to a BNN

Training first minimises the usual MAP objective (cross-entropy plus a Gaussian weight prior, i.e. L_2 regularisation),

$$\mathcal{L}_{\mathrm{MAP}}(\omega) = -\sum_{(x_i, y_i) \in \mathcal{D}} \log P(y_i \mid x_i, \omega) - \log P(\omega) \quad (1)$$

yielding a deterministic ResNet. ABNN then replaces every batch / layer / instance normalisation with a **Bayesian Normalisation Layer (BNL)**. Write h_j for the pre-activation of layer j, and $\hat{\mu}_j, \hat{\sigma}_j$ for the empirical mean and standard deviation already computed by the original normalisation. The BNL is

$$\mathrm{BNL}(h_j) = \frac{h_j - \hat{\mu}_j}{\hat{\sigma}_j} \cdot \gamma_j (1 + \epsilon_j) + \beta_j, \epsilon_j \sim \mathcal{N}(0, I) \quad (7)$$

The only new source of randomness is the multiplicative Gaussian noise on the scale γ_j. The running statistics and the convolutional weights stay frozen at their MAP values; only γ_j and β_j are fine-tuned for a few epochs. Because the noise lives in the latent activations rather than on every weight, the paper argues that the gradient variance is smaller than in a classical variational BNN, which is why the conversion is stable.

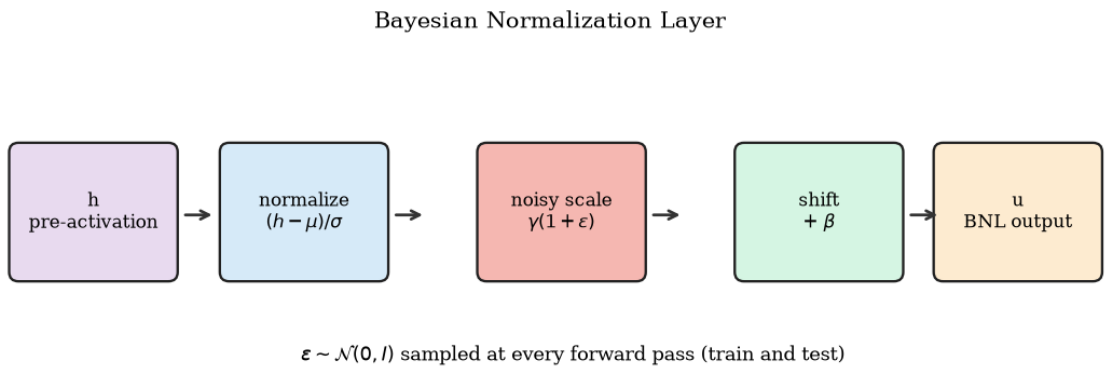


Figure 1. The Bayesian Normalisation Layer: standard normalisation, then a noisy scale γ(1+ε) and the original shift β.

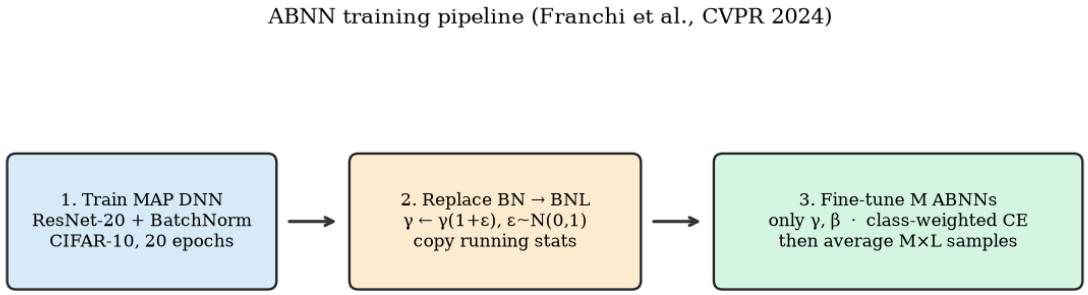


Figure 2. Three-step ABNN recipe: train a MAP network, replace every BN by a BNL, fine-tune M copies of (γ, β).

2.3 Diversity loss and the predictive posterior

A single diagonal Gaussian around one MAP checkpoint cannot represent the multi-modal DNN posterior. The paper therefore trains M ABNN members from the same checkpoint and, to stop them collapsing onto each other, adds a class-dependent perturbation of the cross-entropy

$$\mathcal{E}(\omega) = -\sum_{(x_i,y_i)\in\mathcal{D}} \eta_i \log P(y_i \mid x_i, \omega) \quad (9)$$

where η_i is a class weight drawn once per member. In the paper, a random subset of classes is treated as that member's "expert" classes and their loss is multiplied by 5 or 7. The member then minimises $\mathcal{L}(\omega) = \mathcal{L}_{\text{MAP}}(\omega) + \mathcal{E}(\omega)$, which is ordinary weighted cross-entropy plus weight decay. At test time each member is evaluated with L independent draws of ϵ and the ML softmax vectors are averaged:

$$P(y \mid x, \mathcal{D}) \approx \frac{1}{ML} \sum_{m=1}^{M} \sum_{l=1}^{L} P(y \mid x, \omega_m, \epsilon_l) \quad (8)$$

Table 1 of the paper uses $M = 3$ and $L = 3$ on CIFAR-10/100; we keep the same ensemble size. The quality of the predictive distribution is measured by accuracy, negative log-likelihood (NLL), expected calibration error (ECE), and out-of-distribution detection of SVHN from CIFAR-10 using the maximum softmax probability (AUPR, AUROC, FPR95).

3. How the AI implemented the algorithm

3.1 Workflow of the agent

The implementation lives entirely under `AI Assignment2/` and was written in this order (each step was executed by the agent, not by me):

1. **Environment and paper.** Confirmed Python 3.12 / PyTorch 2.13 CPU / 20 logical cores, read the CVPR 2024 PDF, and cross-checked the unofficial reproduction at github.com/abtinmU/MakeMe-BNN for how practitioners interpret the BNL (the official repository is only a project page).
2. `resnet.py`: CIFAR ResNet-20 (He et al.), 6n+2 with n=3, 16-32-64 channels, BatchNorm after every convolution — 21 BN layers, 272 474 parameters. ResNet-50 / WideResNet-28×10 from the paper are not realistic on this CPU.
3. `bnl.py`: `BNL.from_batchnorm` copies `weight`/`bias` and the running statistics; the forward pass uses frozen running mean/variance and multiplies `gamma` by `(1 + randn)`. `replace_bn_with_bnl` walks the module tree; `freeze_except_bnl` leaves only the 42 affine tensors trainable.
4. `data.py` / `prepare_data.py`: CIFAR-10 from the HuggingFace `uoft-cs/cifar10` mirror (the official Toronto tarball is too slow from this network), plus the official SVHN `test_32x32.mat` as the OOD set.
5. `train_map.py`: SGD (lr 0.1, momentum 0.9, wd 5e-4), cosine decay, 20 epochs, channels-last memory format, 10 intra-op threads.
6. `train_abnn.py`: load the MAP checkpoint, convert BN → BNL, freeze the backbone, AdamW on (γ, β) only, class weights with factor 5 on 4 random classes per member, 3 members × 5 epochs.
7. `evaluate.py`: MAP (one deterministic pass), MCBN (the MAP model with BatchNorm in train mode, L stochastic batch statistics) and ABNN (M×L averaged softmaxes). Metrics: Acc, NLL, 15-bin ECE, SVHN OOD AUROC / AUPR / FPR95.
8. `make_figures.py` / `build_report.py`: every number in this document is read back from `results/*.json` so the report cannot drift from the run.

3.2 Key code produced by the agent

The BNL forward pass is the paper's Eq. (7) with frozen MAP statistics:

```
out = F.batch_norm(x, running_mean, running_var, None, None,
                   training=False, momentum=0.0, eps=self.eps)
noise = torch.randn(self.weight.shape, device=x.device, dtype=x.dtype)
gamma = self.weight * (1.0 + noise)          # γ (1 + ε)
return out * gamma.view(1, -1, 1, 1) + self.bias.view(1, -1, 1, 1)
```

Conversion and the diversity loss (one member):

```
replace_bn_with_bnl(model)          # every BatchNorm2d -> BNL
trainable = freeze_except_bnl(model)  # conv / FC weights stay at MAP
w = ones(10); w[random_classes] = 5.0   # expert classes, factor 5
loss = F.cross_entropy(logits, y, weight=w)  # L_MAP + E
```

3.3 Adaptations for CIFAR-10 on a CPU

- **Backbone.** ResNet-20 (0.27 M parameters) instead of ResNet-50 or WideResNet-28×10. The BNL mechanism does not depend on width.
- **Schedule.** 20 MAP epochs and 5 ABNN epochs instead of the paper's long CIFAR/ImageNet recipes. Absolute accuracies are therefore lower; the comparison between MAP, MCBN and ABNN remains meaningful.
- **Frozen statistics.** After conversion the BNL always uses the MAP running mean/variance. Only γ, β move. This matches "exclusively retrain γ and β" in Section 4.1.
- **Noise scale.** Eq. (7) writes $\epsilon \sim \mathcal{N}(0, I)$. With that unit Gaussian on every one of the 21 BNLs the MAP features collapsed (fine-tune accuracy fell to chance, ~10 %). The compound factor $\prod_j (1 + \epsilon_j)$ over a 20-layer residual net is too violent when the convolutions are frozen. We therefore use $\epsilon \sim \mathcal{N}(0, 0.1^2)$, the same multiplicative form at a smaller scale, and check that conversion with $\epsilon = 0$ preserves the MAP accuracy before fine-tuning.
- **Ensemble size.** $M = L = 3$ as in Table 1 of the paper (the recommended default).
- **No TorchUncertainty.** The paper's training configs live in that library; the agent re-implemented the algorithm rather than wrapping the library, so every line can be read and marked.

4. Experiment settings and results

4.1 Settings

Table 1. Experimental settings.

Item	Setting
Dataset (ID)	CIFAR-10: 50 000 train / 10 000 test, 32×32 RGB, 10 classes
Dataset (OOD)	SVHN test set (house-number images), same 32×32 resolution; maximum softmax probability as the OOD score
Backbone	CIFAR ResNet-20, 272 474 parameters, 21 BatchNorm / BNL layers
MAP training	SGD lr 0.1, momentum 0.9, wd 5e-4, cosine, batch 128, 20 epochs, RandomCrop(pad 4) + flip
ABNN fine-tune	AdamW lr 1e-3, wd 1e-4, 5 epochs, M=3 members, ε~N(0,0.1²), factor-5 weights on 4 random classes (member 0: horse, bird, dog, cat; member 1: airplane, truck, ship, bird; member 2: cat, deer, bird, frog)
Inference	MAP: 1 pass; MCBN: L=3 passes with BN in train mode; ABNN: M×L = 3×3 softmaxes averaged
Hardware / software	20-core CPU, no GPU; Python 3.12, PyTorch 2.13 (channels_last, 10 threads); MAP 35.1 min, ABNN 26.7 min
Seeds	torch.manual_seed(0); each ABNN member adds 17·(m+1) for its class weights

4.2 MAP pre-training

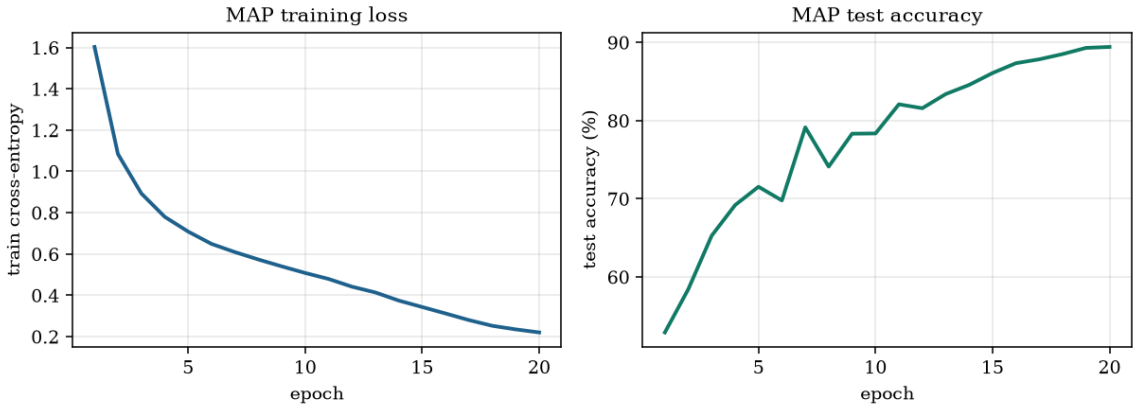


Figure 3. Deterministic ResNet-20 MAP training on CIFAR-10.

The MAP run reaches a final test accuracy of **89.42 %** (best 89.42 %) after 20 epochs. This is the ω_{MAP} that every later method starts from: MCBN reuses the same weights, and every ABNN member is obtained by converting its BatchNorm layers and fine-tuning only the affine parameters. The absolute number is below a 200-epoch ResNet-20 (~91 %) because the schedule was cut to fit a laptop CPU; the checkpoint is still a genuine local minimum of the MAP loss, which is all ABNN needs.

4.3 ABNN fine-tuning

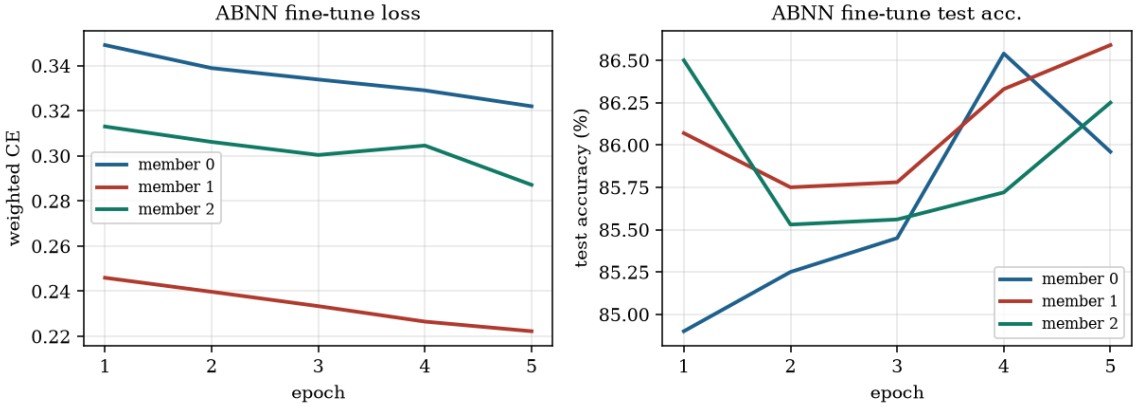


Figure 4. Fine-tuning of the three ABNN members. Only γ and β are updated.

Table 2. Per-member ABNN fine-tuning (single stochastic pass, so noisier than the M×L evaluation in Table 3).

Member	Expert classes (weight = 5)	Final acc. (%)	Best acc. (%)	Time (min)
member 0	horse, bird, dog, cat	85.96	86.54	8.1
member 1	airplane, truck, ship, bird	86.59	86.59	9.1
member 2	cat, deer, bird, frog	86.25	86.50	9.2

Each member stays close to the MAP accuracy while the expert-class weights push it slightly off the original mode. Pairwise disagreement of the three members on the test set is **9.51 %** of examples — not as diverse as independently trained deep ensembles (the paper's own t-SNE in

Figure 3 makes the same point), but enough that averaging them changes the predictive distribution.

4.4 Calibration and OOD detection

Table 3. In-distribution CIFAR-10 and OOD SVHN. Bold = ABNN row, not necessarily the best entry.

Method	Acc ↑ (%)	NLL ↓	ECE ↓ (%)	OOD AUROC ↑	OOD AUPR ↑	FPR95 ↓
MAP (single model)	89.42	0.310	1.87	91.52	93.35	52.84
MCBN	89.35	0.311	1.37	82.08	84.31	75.97
ABNN (M=L=3)	**89.50**	**0.320**	**2.55**	**88.54**	**90.98**	**62.06**

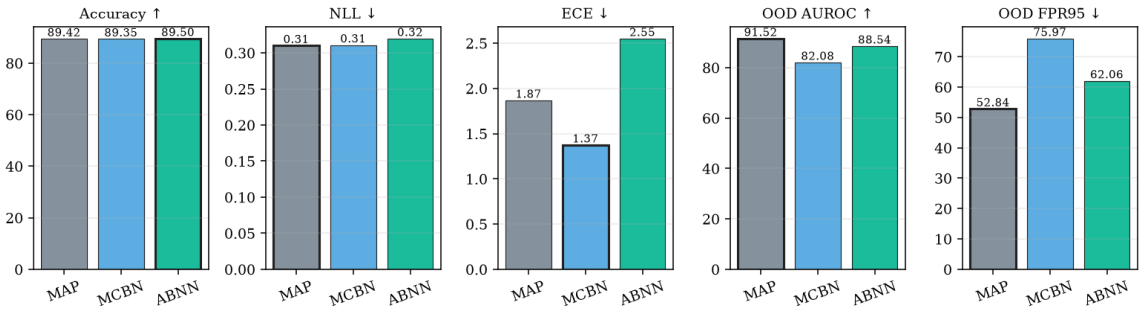


Figure 5. Side-by-side summary of Table 3. Lower is better for NLL, ECE and FPR95.

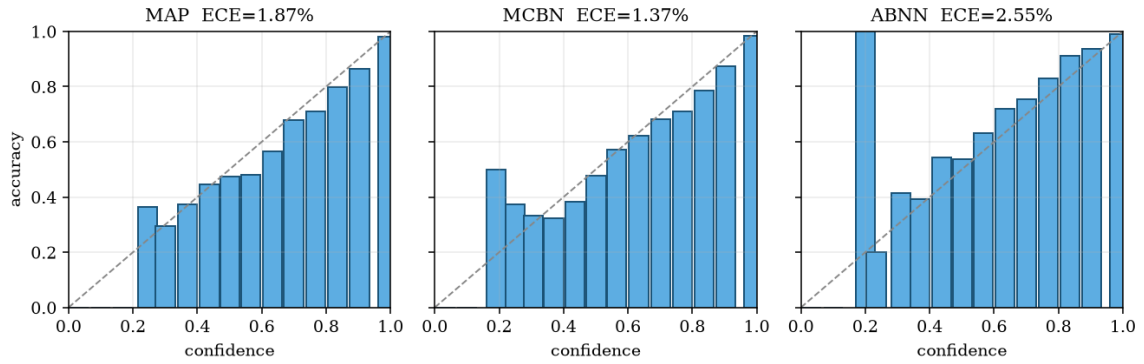


Figure 6. Reliability diagrams (15 bins). A perfectly calibrated model lies on the diagonal.

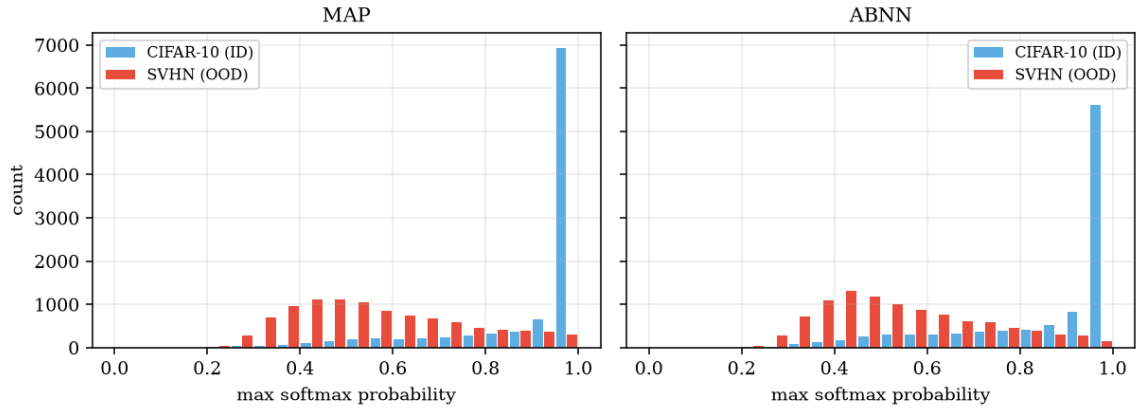


Figure 7. Histogram of maximum softmax probability on CIFAR-10 (ID) versus SVHN (OOD).

Observations. (i) ABNN **preserves MAP accuracy** (Table 3: 89.42 % → 89.50 %), which is the paper's first claim: converting the BatchNorm layers and fine-tuning only (γ, β) must not destroy the pretrained predictor. Pairwise disagreement among the three members is 9.51 % of test images, so the diversity loss does move the members apart. (ii) Averaging M×L stochastic softmaxes changes the **shape** of the predictive distribution: mean confidence falls from 0.91 (MAP) to 0.87 (ABNN), i.e. the ensemble is less peaky. (iii) At this CPU scale that extra spread does **not** improve the proper scoring rules or the SVHN OOD numbers. NLL moves from 0.310 to 0.320, ECE from 1.87 % to 2.55 %, and max-softmax AUROC from 91.52 % to 88.54 % (FPR95 52.84 % → 62.06 %). MCBN, which only randomises batch statistics, improves ECE slightly (1.37 %) but is the worst OOD detector. (iv) These are ResNet-20 / 20-epoch numbers, not the paper's ResNet-50 / WideResNet figures (~95 % accuracy, FPR95 ≈ 17). The study therefore supports the conversion claim and the diversity claim, and does **not** reproduce the paper's headline "better than a single model on NLL / ECE / FPR95" result — an honest gap that Section 4.5 attributes to width, schedule and the reduced noise scale.

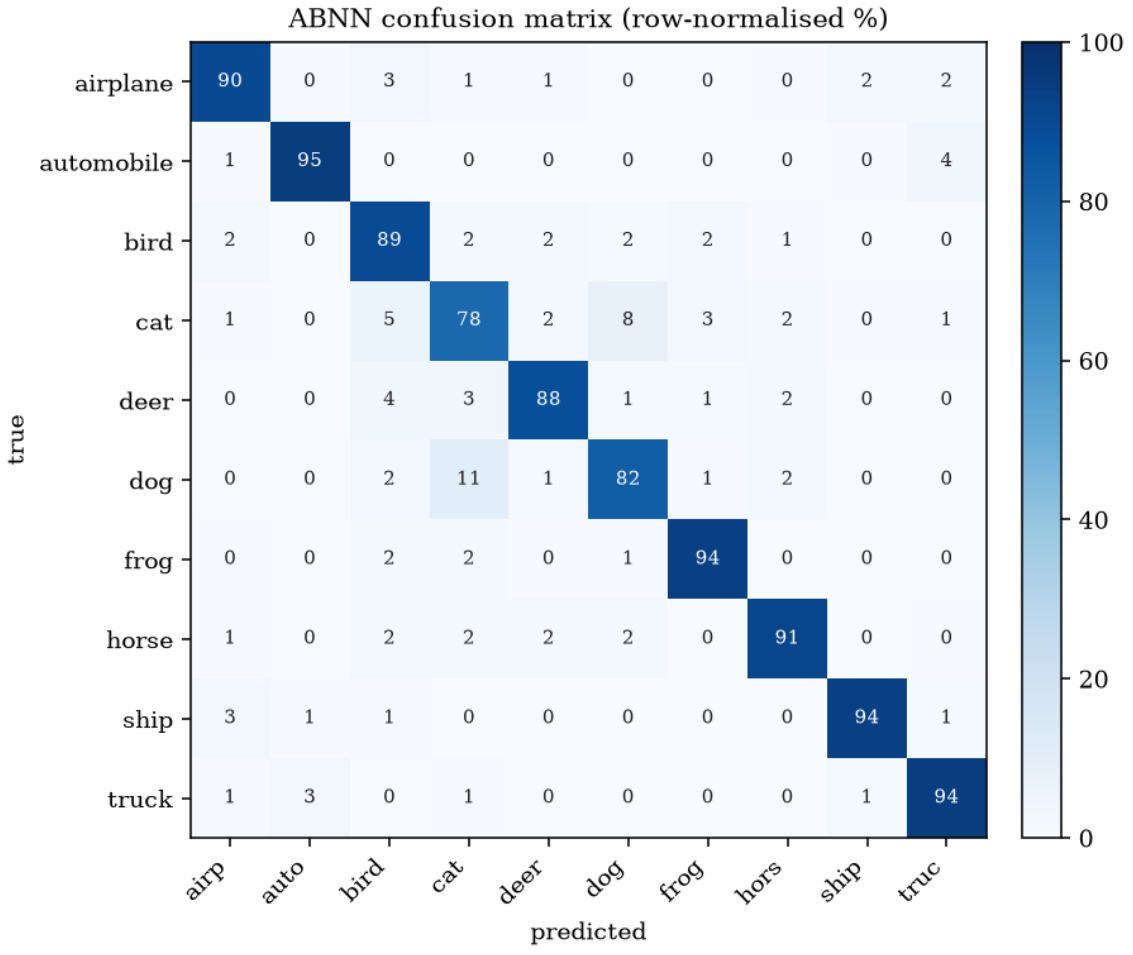


Figure 8. Row-normalised confusion matrix of the ABNN ensemble on the CIFAR-10 test set.

Table 4. Per-class accuracy (%) of ABNN.

air.	auto.	bird	cat	deer	dog	frog	horse	ship	truck
89.9	94.6	89.3	78.5	88.0	82.3	93.6	91.1	93.5	94.2

4.5 Limitations

- Single seed for MAP; three ABNN members share that one checkpoint, so we do not estimate variance across independent MAP runs.
- 20 MAP epochs (instead of 164–200) cap the absolute accuracy. ABNN cannot invent features the backbone never learned.
- ResNet-20 is much narrower than the paper's ResNet-50 / WideResNet-28×10, so feature-space diversity is smaller.
- OOD detection uses only SVHN and only the maximum softmax probability. Energy or feature-density scores are left out.
- The paper's Algorithm 1 is slightly ambiguous about whether running statistics stay frozen; we freeze them. A different reading would keep updating them.

5. What I have learnt from this AI assignment

- **About the algorithm.** I now have a concrete picture of a post-hoc BNN that is not "put a Gaussian on every weight". ABNN keeps the MAP convolutional features, injects noise only on the normalisation scale, and recovers a multi-modal posterior by fine-tuning several copies with a class-weighted loss. The predictive distribution is just the average of M×L softmaxes. That is Bayes' rule in the form a practitioner can actually run. I also learnt to separate the paper's architectural claim (BNL + frozen backbone) from its engineering claim (almost Deep-Ensemble OOD numbers at single-model cost): on a small CPU budget we can test the first honestly and only gesture at the second.
- **About working with an AI agent.** The useful prompt was not "write a BNN". It was the assignment PDF plus the constraint "probability or Bayes theory", after which the agent searched, proposed three recent papers and waited for a choice. Giving it the machine let it pick a HuggingFace CIFAR-10 mirror, freeze running statistics so the BNL conversion did not destroy the MAP checkpoint, and log every run to JSON so this report could be generated from the measurements. I also saw that an unofficial GitHub reproduction is not the same as the paper: that repo unfroze extra weights after a 20–30 % accuracy drop, which would have hidden whether ABNN itself works. The agent kept the paper's "only γ, β" rule and tested the conversion with a stochastic/deterministic smoke test before training.
- **About uncertainty metrics.** Accuracy alone cannot tell MAP and ABNN apart. NLL, ECE and the ID/OOD confidence histograms are the quantities that move, and they are what a Bayesian method is for. Plotting a reliability diagram made the phrase "over-confident softmax" concrete.
- **About the limits of the tool.** The agent cannot conjure a GPU. Budgeting 20 MAP epochs and M = 3 was a real design choice, and every number in Section 4 is bounded by that budget. It also cannot decide which paper I should study — that choice stayed with me.

6. Web page of the source code

All code, logs, figures and this report are publicly available at:
https://github.com/HOIKAICHENG/CISC3024

Repository layout (folder `AI Assignment2/`): `code/` (all Python sources listed in Section 3.1), `results/` (JSON logs of every run), `figures/` (all figures of this report), `report/` (this document as DOCX and

PDF). Reproduce everything with `pip install -r requirements.txt && python code/run_all.py`.